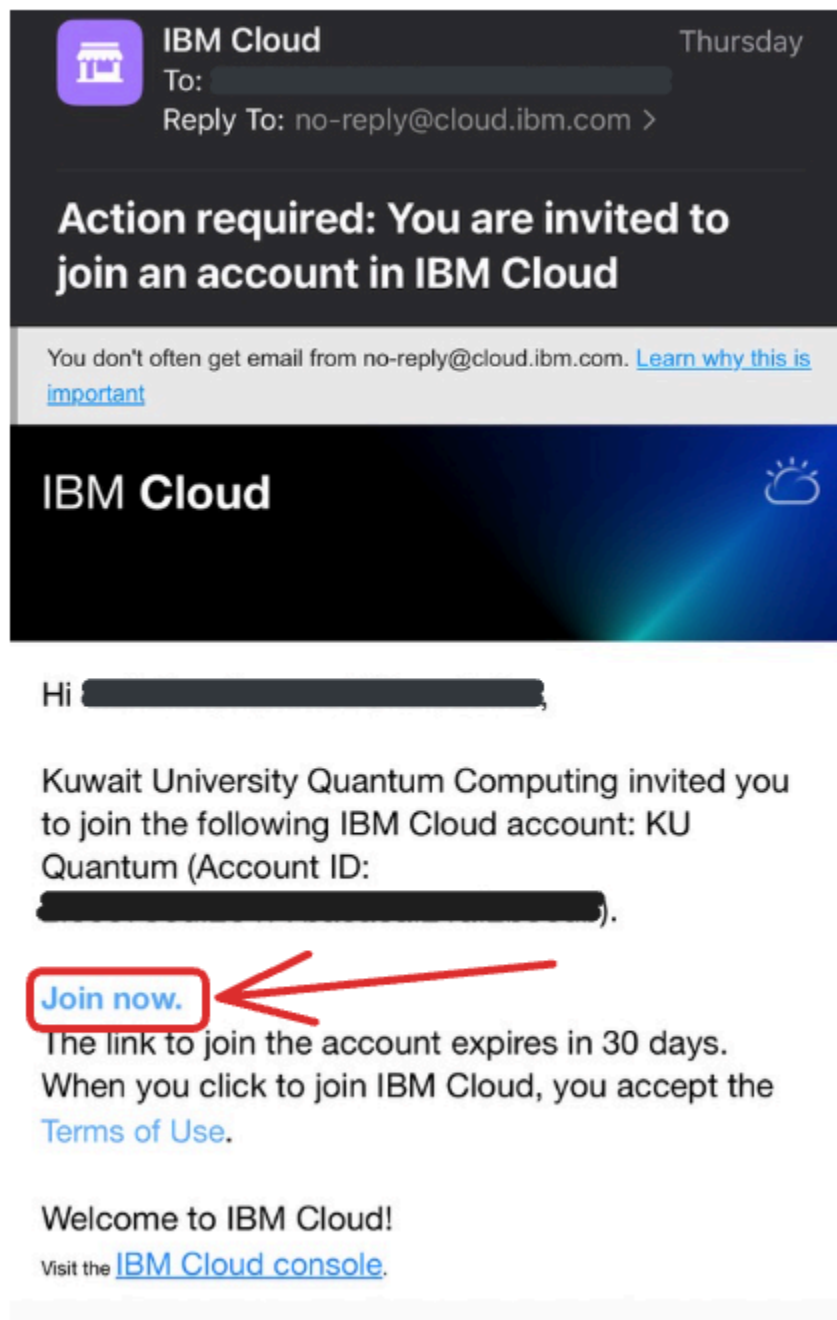
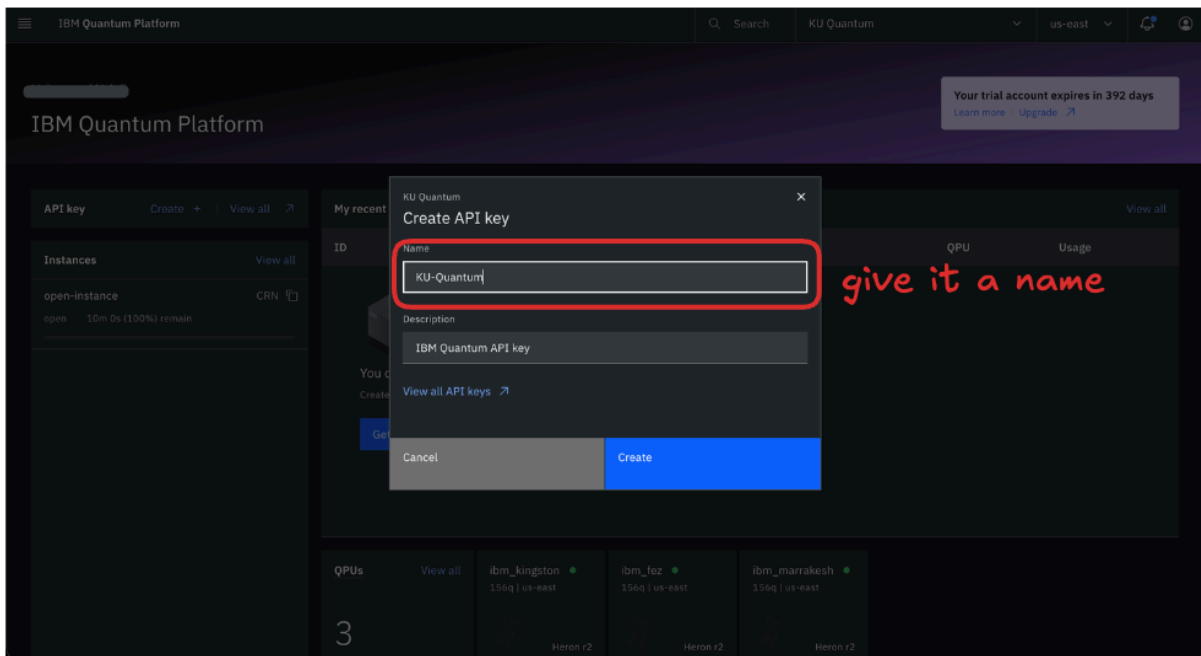
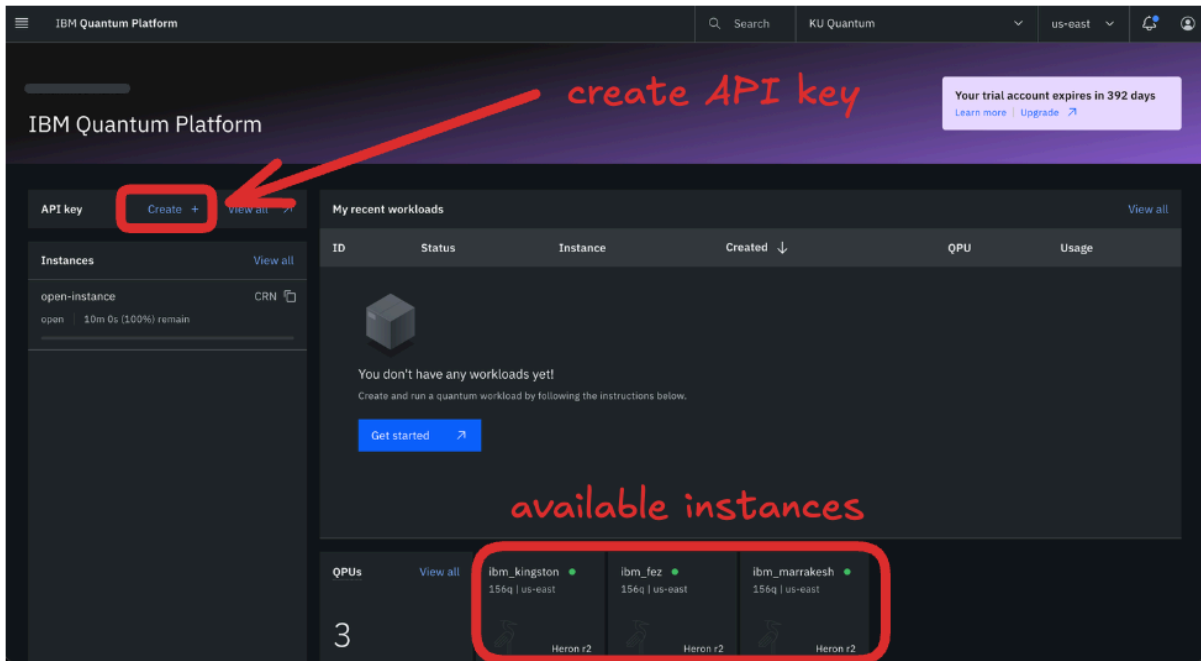


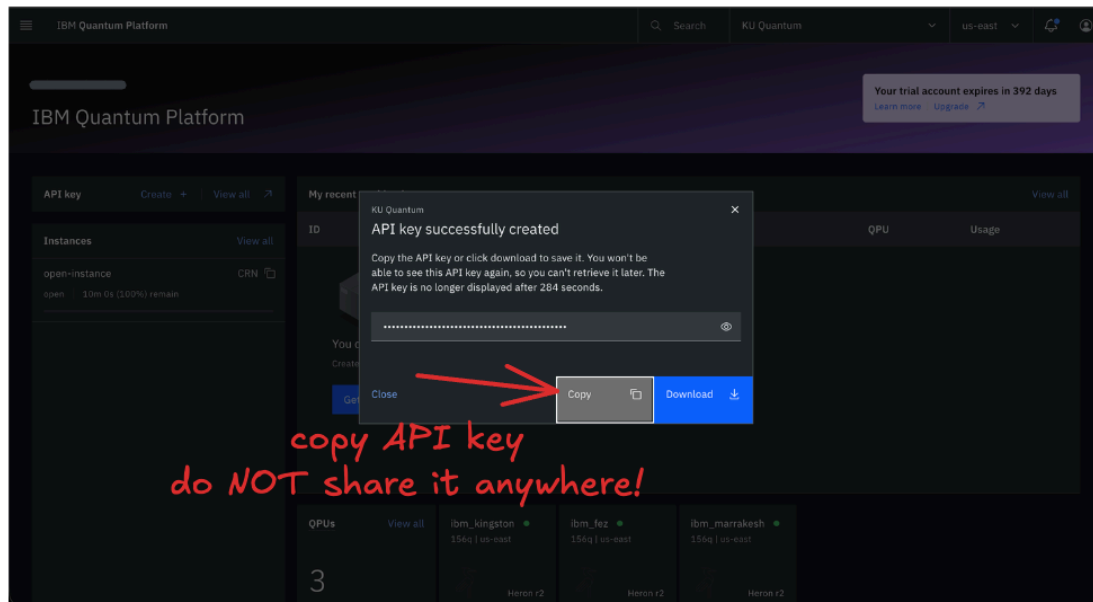
IBM Quantum Cloud “Hello World”

1. Follow the invitation link in your mailbox. Create your account and complete the questionnaire.



2. Create an API key for your account. It is important that you **DO NOT** share this key with anyone. Note the available instances at the bottom, you can select any of them later on.



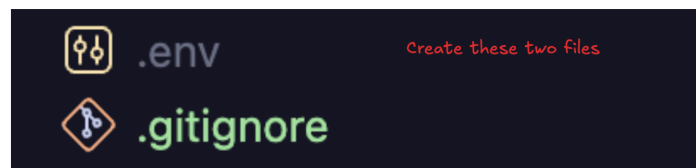


- From this point, we'll assume you've set up your environment as instructed in **HW 1**. You will need to install `dotenv` through your terminal: `pip install python-dotenv`

You can also follow along by cloning the Github repo I've set up. The IBM "Hello world" example is outdated and has broken lines. You can find the repository here:

github.com/Cyberistic/Quantum-Computer-IBM-Setup-Tutorial

- In your project root, create two new empty folders, `.gitignore` and `.env`



In your `.gitignore` file, add `".env"` as a line, this will make sure your API key is never pushed to git.



In your `.env` file, add `IBM_QUANTUM_TOKEN=<api_key_you_copied>` and `IBM_QUANTUM_BACKEND=<one_of_the_instances_optionally>`

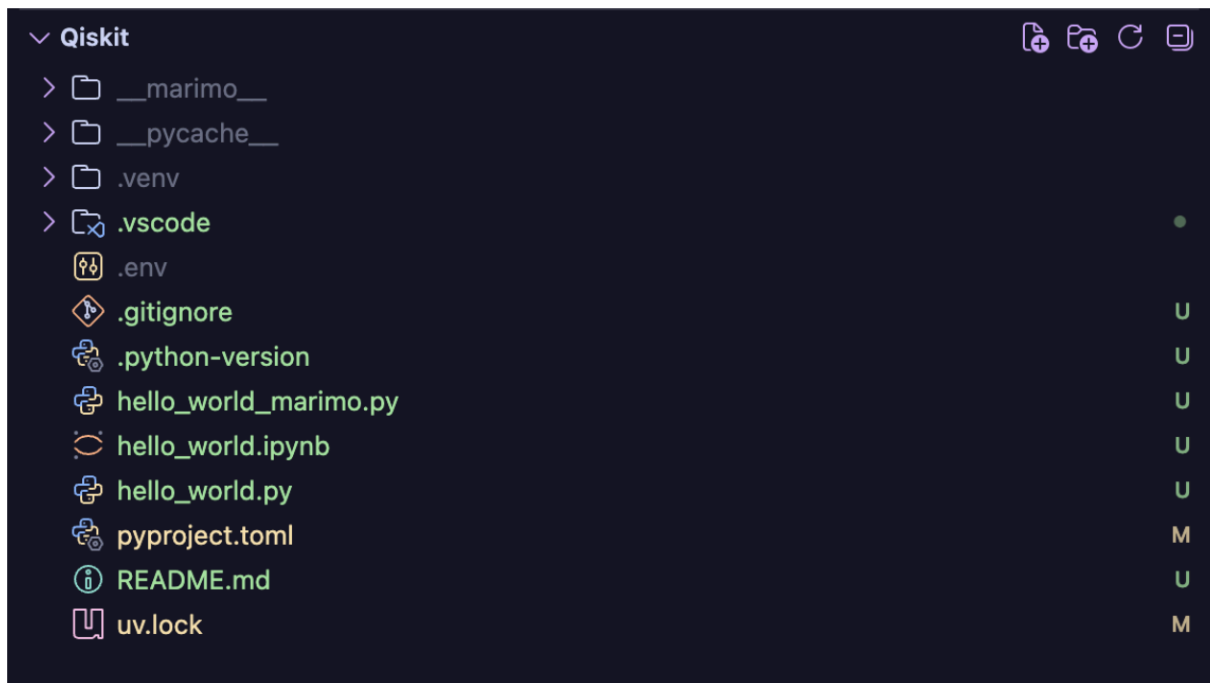
Replace with your actual keys (from step 2). It is very crucial that you do not share this file, think of it as containing your password!

```
.env
6  IBM_QUANTUM_TOKEN=FAKE-A-KEYHERENICETRYOUTHUGHTIWOULDSHARE??
5
4  # Optional: pin a specific backend. Comment out to auto-select via least_busy.
3  IBM_QUANTUM_BACKEND=ibm_kingston
2
1
7  |
```

paste your full key here

5. Now you are ready to use your account, If you are using the github repository provided, your folder should look something like this:

If you're not sure how to set up the virtual environment, reference **HW 1** or the provided [README.md](#) file (or ask your agent friend to set it up for you!)



The screenshot shows a file explorer for a project named 'Qiskit'. The files and folders listed are: `__marimo__`, `__pycache__`, `.venv`, `.vscode`, `.env`, `.gitignore`, `.python-version`, `hello_world_marimo.py`, `hello_world.ipynb`, `hello_world.py`, `pyproject.toml`, `README.md`, and `uv.lock`. The status of each file is indicated by a letter on the right: `.env` is 'U' (Untracked), `.gitignore` is 'U' (Untracked), `.python-version` is 'U' (Untracked), `hello_world_marimo.py` is 'U' (Untracked), `hello_world.ipynb` is 'U' (Untracked), `hello_world.py` is 'U' (Untracked), `pyproject.toml` is 'M' (Modified), `README.md` is 'U' (Untracked), and `uv.lock` is 'M' (Modified).

Side note: the project is set up with `uv` instead of `pip` (because it's not 2020 anymore). Additionally, I use `marimo` instead of `jupyter`, but I've provided the `jupyter` and `.py` versions for you to decide what to use. All output is tested and should be the same

6. We set up `.env` so that you do not have to hardcode your keys inside the code. This makes it easier to share your project in submissions or through github. To activate the environment, run the first two blocks:

1. Load credentials from `.env`

Credentials come from `.env` (`IBM_QUANTUM_TOKEN`), and optionally `IBM_QUANTUM_BACKEND`. Do NOT hardcode your token in source code or commit your `.env` file.

```
# Resolved relative to this file so the notebook works from any working
# directory, not just the project root.
load_dotenv(dotenv_path=Path(__file__).parent / ".env")
token: str | None = os.environ.get(key="IBM_QUANTUM_TOKEN")
pinned: (str & ~AlwaysFalsy) | None = os.environ.get(key="IBM_QUANTUM_BACKEND") or None
```

✓ 0.0s Python (marimo)

2. Save the account (one-time)

`QiskitRuntimeService.save_account()` writes the credentials to disk so later sessions authenticate automatically. Once you run it, you don't need to call this cell every time.

```
QiskitRuntimeService.save_account(
    # For `token`, use the 44-character API_KEY you created
    # and saved from the IBM Quantum Platform Home dashboard
    token=token,
    overwrite=True,
)
```

✓ 0.0s Python (marimo)

3. Authenticate

Run this every time you need the service. With credentials already saved, `QiskitRuntimeService()` picks them up with no arguments.

```
service = QiskitRuntimeService()
print("Connected to:", service.channel)
```

✓ 4.3s Python (marimo)

Connected to: ibm_quantum_platform
qiskit_runtime_service.__init__:WARNING:2026-10-04 01:41:40,748: Instance was not set at service ins

7. Now you are ready to locally build your “Hello world” circuit, note the simulated state-vectors and the value we expect to get once we run on the IBM quantum computer:

4. Build the circuit

Simple Bell pair: A Hadamard on qubit 0 then a CNOT onto qubit 1

```
qc = QuantumCircuit(2)
qc.h(qubit=0)
qc.cx(control_qubit=0, target_qubit=1)
```

✓ 0.0s

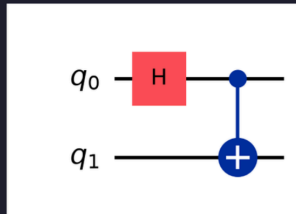
Python (marimo)

<qiskit.circuit.instructionset.InstructionSet object at 0x1191f0b20>

```
qc.draw(output="mpl")
```

✓ 1.3s

Python (marimo)



```
# Ideal state vector, computed locally - no hardware needed.
# Basis order is |00>, |01>, |10>, |11>.
sv: Statevector = Statevector.from_instruction(instruction=qc)
print(f"Statevector: {sv.data}")
print(f"Probabilities: {dict(sv.probabilities_dict())}")
```

✓ 0.0s

```
Statevector: [0.70710678+0.j 0. +0.j 0. +0.j 0.70710678+0.j]
Probabilities: {np.str_('00'): np.float64(0.4999999999999999), np.str_('11'): np.float64(0.4999999999999999)}
```

Observable

$\langle Z \otimes Z \rangle$ on the Bell pair. Expected value is +1 for $|\Phi^+\rangle$ — the qubits are perfectly correlated.

```
observable = SparsePauliOp(data="ZZ")

# The exact noiseless answer, for comparison against the hardware result.
print(f"Ideal <ZZ> (exact): {sv.expectation_value(oper=observable).real}")
```

✓ 0.0s

```
Ideal <ZZ> (exact): 0.9999999999999998
```

We are expecting a value close to this

8. Prepare it to be sent to the Quantum Computer:

5. Transpile to the backend

Maps the circuit and the observable onto the backend's physical qubits.

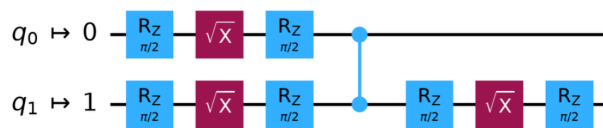
```
pm: Unknown = generate_preset_pass_manager(backend=backend, optimization_level=1)
isa_circuit: Unknown = pm.run(qc)
isa_observable: SparsePauliOp = observable.apply_layout(isa_circuit.layout)

isa_circuit.draw("mpl")
```

✓ 0.1s

Python (marimo)

Global Phase: $3\pi/4$



9. Finally, you are ready to send it to the cloud quantum computer and get your results. Note that this run will cost you compute points (and you are allocated a limited amount). You might need to wait in the queue depending on your chosen backend.

6. Run the estimator

Submits to the backend and waits for the result. This consumes real quantum-compute time on your account.

```
estimator = Estimator(mode=backend)
job: RuntimeJobV2 | LocalRuntimeJob = estimator.run(pubs=[(isa_circuit, isa_observable)])
result: Any | PrimitiveResult[Unknown] | QuantumP... = job.result()
```

✓ 1m 14.0s

qiskit_runtime_service.backends.WARNING:2026-10-04 01:54:27,742: Using instance: open-instance, plan: open

```
# ty: ignore[unresolved-attribute]
print(f"Expectation value: {result[0].data.evs}")
```

✓ 0.0s

Expectation value: 1.0043937384735346

success!
within % error

Expectation value

Should be $\approx +1$ — confirming the Bell pair is in the $|\Phi^+\rangle$ state with perfect Z -parity correlation.

```
# ty: ignore[unresolved-attribute]
result[0].data.evs
```

✓ 0.0s

array(1.00439374)

You can check your usage through the dashboard:

Instances		View all
open-instance		CRN
open	8m 51s (89%) remain	